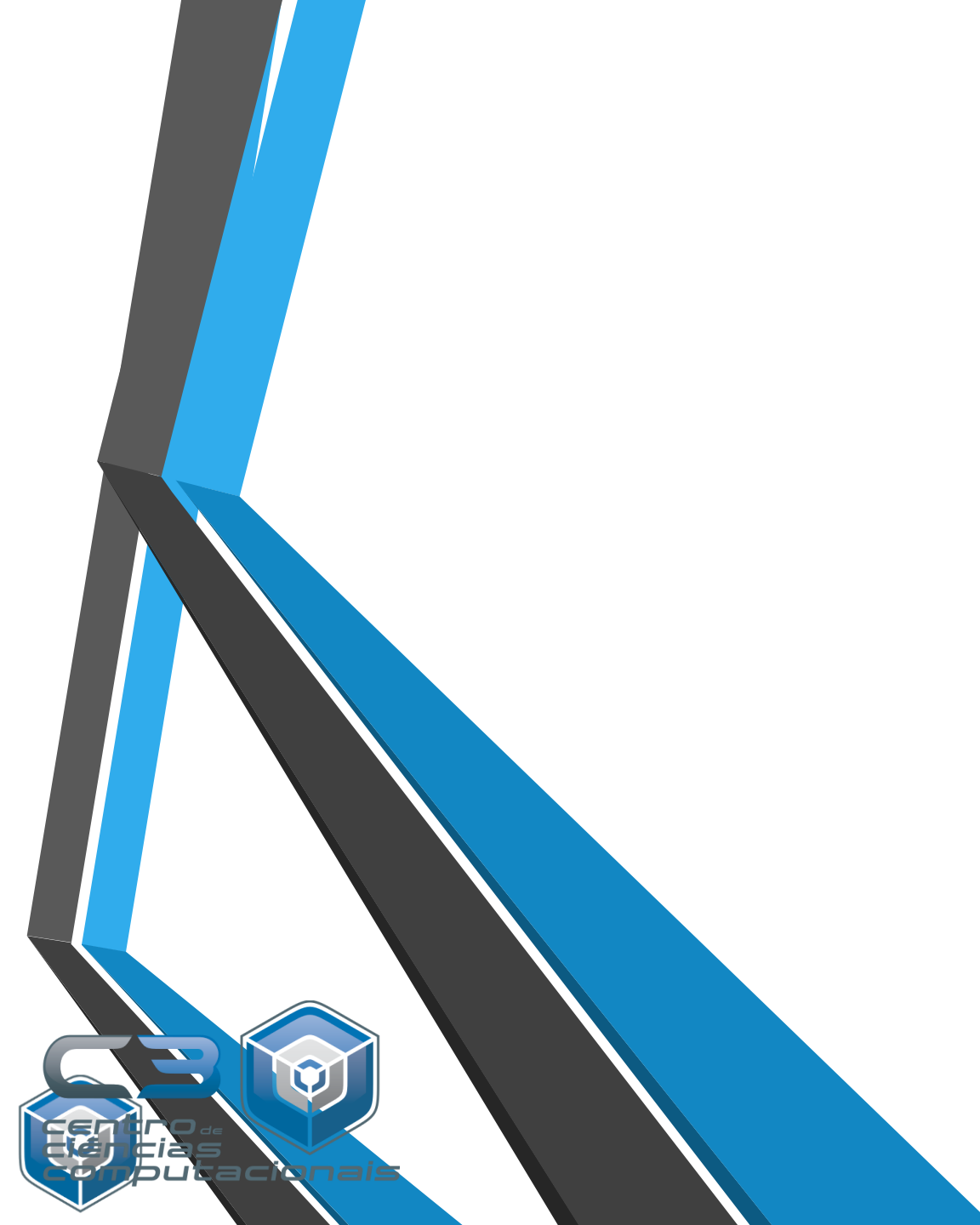


Recursividade

Matemática Discreta

Prof. Emanuel Estrada



Introdução

- A **recursividade** é uma forma de resolver problemas por meio da divisão dos problemas em subproblemas menores de mesma natureza.
- Em matemática, pode ser definida como: o ato de definir um objeto (geralmente uma função), em termos do próprio objeto
- Em computação, ocorre quando: um dos passos de um determinado algoritmo envolve a repetição desse mesmo algoritmo



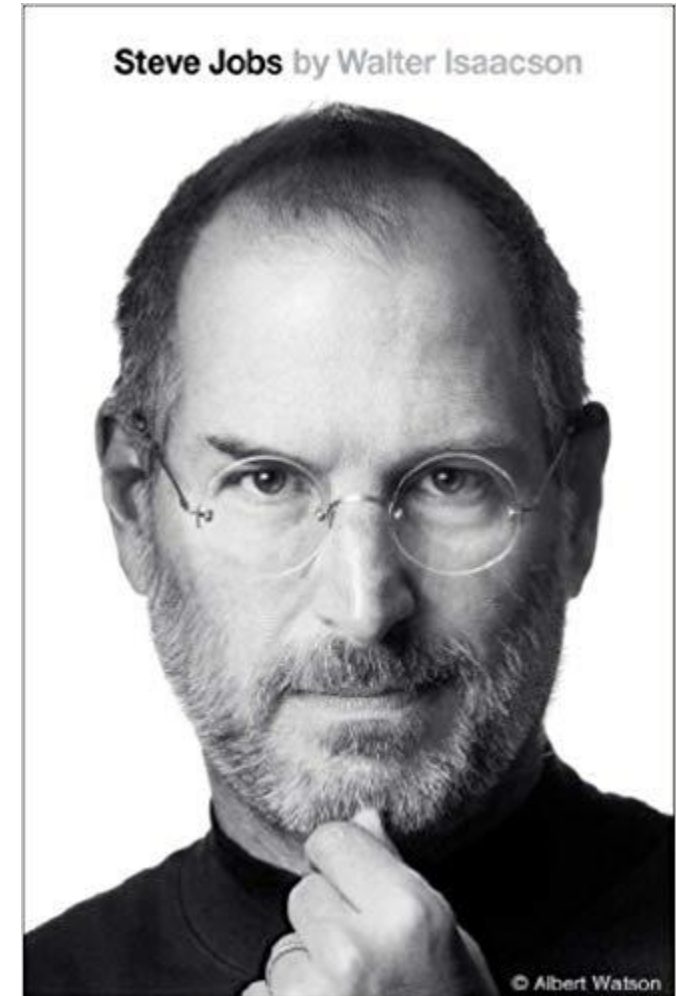
Recursão - Definição

- **Caso base:** Parte não recursiva, também chamado de âncora. Ocorre quando a **resposta** para o problema é **trivial**
- **Passo recursivo:** Parte da definição que especifica como cada elemento (solução) é gerado a partir do precedente



Introdução

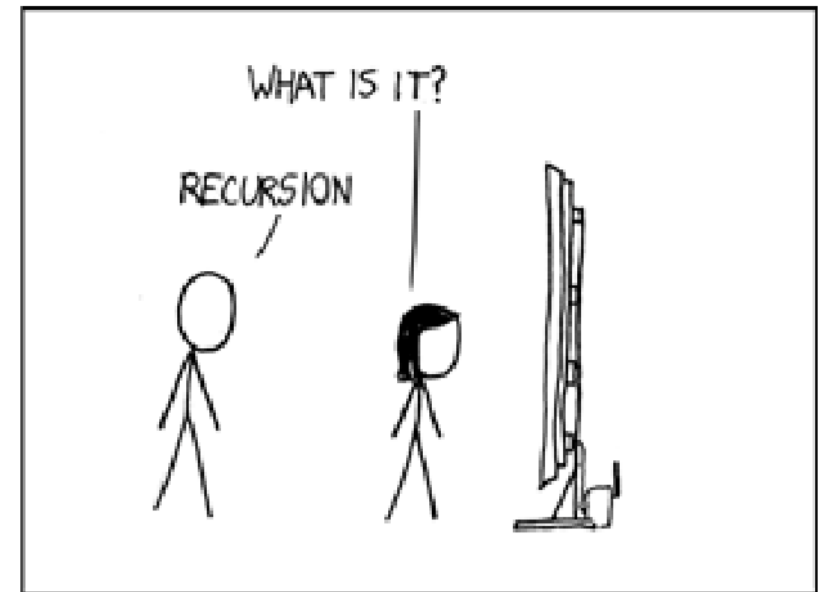
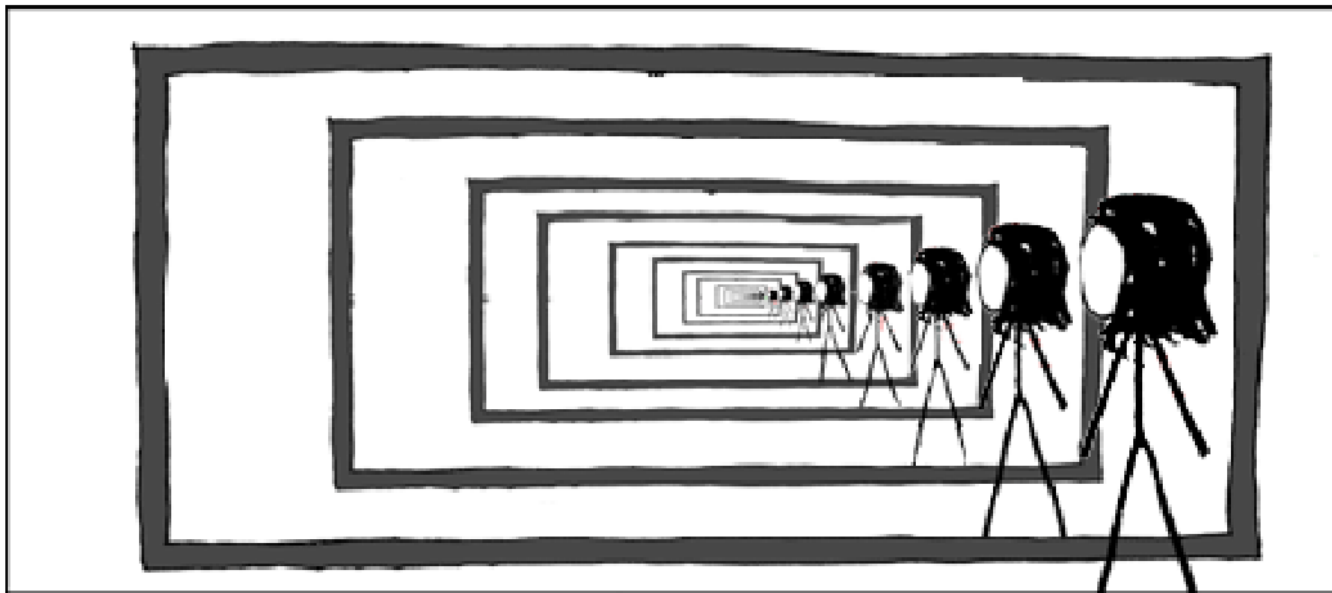
- “Para saber recursividade, é preciso saber recursividade.”



-Bill Gates



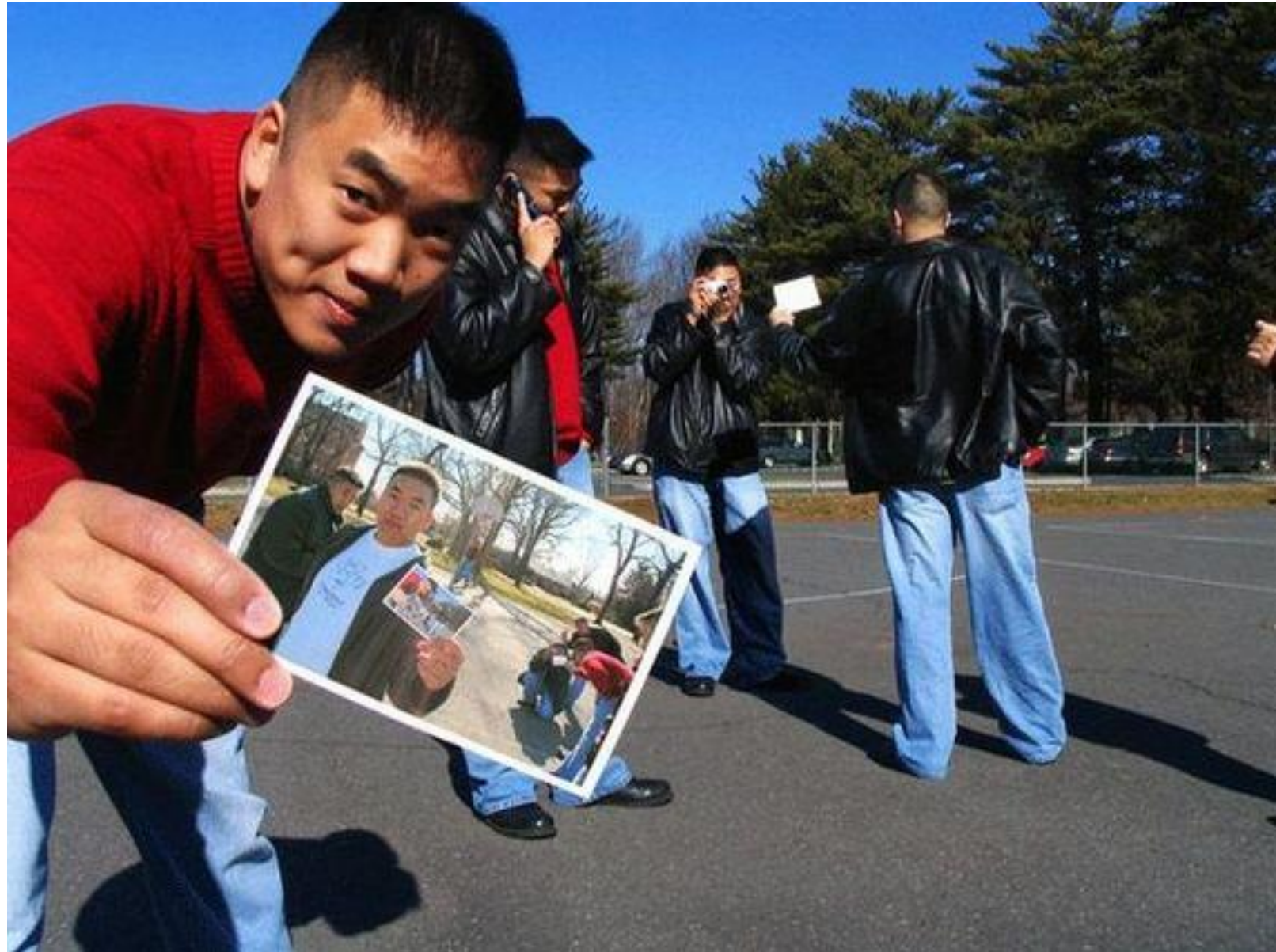
Introdução



Fonte: xkcdsw.com



Exemplo: Efeito Droste



Exemplo: Google



TodasImagensVídeosNotíciasShoppingLivrosWebMaisFerramentas

Ver resultados relacionados

Recursividade

Ciência da computação

Você quis dizer: **recursividade**

Wikipedia

<https://pt.wikipedia.org/wiki/Recursividade>

Recursividade – Wikipédia, a enciclopédia livre

Recursividade (em português europeu: Recorrência), é um termo geralmente usado para descrever o processo de repetição de um objeto de um jeito similar ao que j ...

As pessoas também perguntam :

Qual o conceito de recursividade?

Como entender recursividade?

O que é pensamento recursivo?

Como funciona a recursão?

Feedback

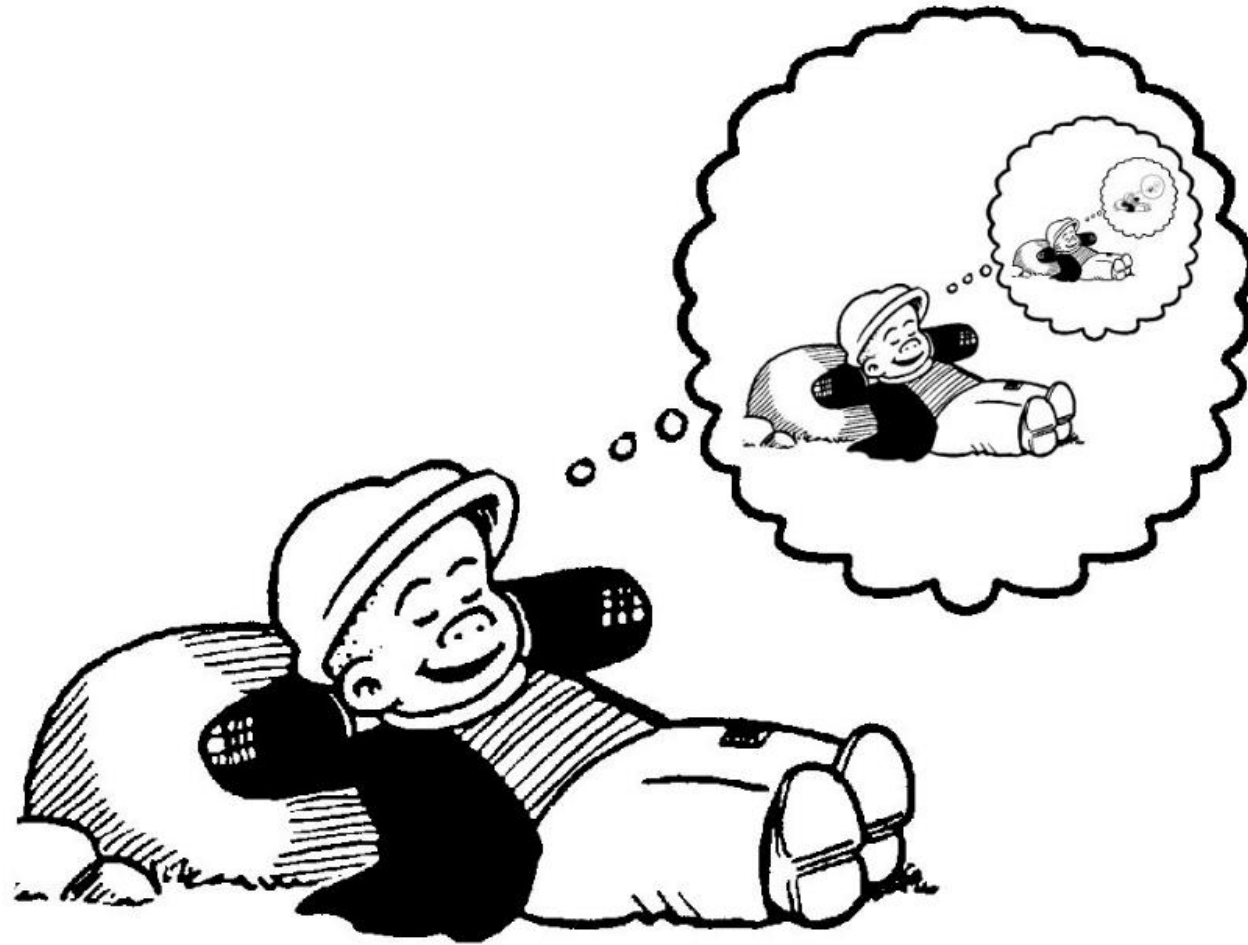
Recursividade :



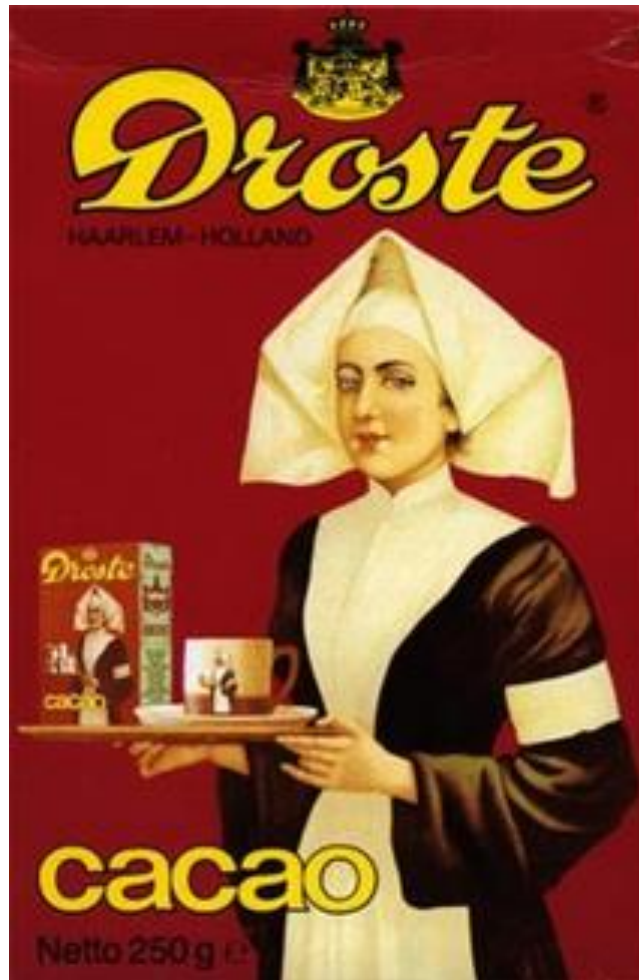
Recursividade, é um termo geralmente usado para descrever o processo de repetição de um objeto de um jeito similar ao que já fora mostrado. Um bom exemplo disso são as imagens repetidas que aparecem quando dois espelhos são apontados um para o outro.

[Wikipédia >](#)

Exemplo: humor



Exemplo: Efeito Droste



Por que Recursividade?

- Presente nas principais linguagens de programação
 - C/C++, Java, Python,...
- Paradigma Funcional
- Essência dos problemas de **divisão e conquista**
- Algoritmos baseados em recursão
 - Quicksort, merge sort, busca binária,...
- Estrutura de dados recursivas
 - Listas encadeadas, árvores binárias,...



Recursividade - Exemplo

- Fatorial de n

$$n! = \begin{cases} 1 & \text{Se } n = 0 \text{ ou } n = 1 \\ n.(n - 1)! & \text{Se } n > 1 \end{cases}$$



Exemplo: Fatorial de n em C

```
double fatrec(long long n)
{
    double res = 1;
    if (n<=1)
        return res;
    else
        return n*fatrec(n-1);
}

void main(int argc, char const *argv[])
{
    double res; int i=56;
    res = fatrec(i);
    printf("fat(%d) = %g", i, res);
}
```



Exemplo: Fatorial de n em C

```
double fatrec(long long n)
{
    double res = 1;
    if (n<=1)
        return res;
    else
        return n*fatrec(n-1);
}

void main(int argc, char const *argv[])
{
    double res; int i=56;
    res = fatrec(i);
    printf("fat(%d) = %g", i, res);
}
```

Caso base



Exemplo: Fatorial de n em C

```
double fatrec(long long n)
{
    double res = 1;
    if (n<=1)
        return res;
    else
        return n*fatrec(n-1);
}

void main(int argc, char const *argv[])
{
    double res; int i=56;
    res = fatrec(i);
    printf("fat(%d) = %g", i, res);
}
```

Passo indutivo



Exemplo: Cálculo Fatorial

- A forma de um compilador implementar um procedimento recursivo é por meio de uma **pilha**
- Nessa pilha são armazenados dados usados em cada chamada de uma função que ainda não terminou de processar

```
Fatorial(4)
[4*fatorial(3)]
[4*<3*fatorial(2)>]
[4*<3*{2*fatorial(1)}>]
[4*<3*{2*1}>]
[4*<3*2>]
[4*6]
[24]
```



Recursividade - Exemplo

- Definição recursiva de 2^n , onde $n \in \mathbb{N}$

$$2^n = \begin{cases} 1 & \text{Se } n = 0 \\ 2 * 2^{(n-1)} & \text{Se } n \geq 1 \end{cases}$$



Recursividade - Exemplo

- Definição recursiva de 2^n , onde $n \in \mathbb{N}$

```
int pot2(int n)
{
    if (n == 0)
        return 1;
    else
        return 2 * pot2(n - 1);
}
```



Recursividade - Exemplo

- Definição recursiva de a^N , onde $a \in \mathbb{Z}$ e $n \in \mathbb{N}$

$$a^N = \begin{cases} 1 & \text{Se } N = 0 \\ a.a^{N-1} & \text{Se } N > 0 \end{cases}$$



Recursividade - Exemplo

- Definição recursiva de a^N , onde $a \in \mathbb{Z}$ e $n \in \mathbb{N}$

```
int pot(int a, int n)
{
    if (n == 0)
        return 1;
    else
        return a * pot(a, n-1);
}
```



Recursividade - Exemplo

- Definição recursiva de $\sum_{k=0}^n a_k$

$$\sum_{k=0}^n a_k = \begin{cases} a_0 & \text{Se } N = 0 \\ a_n + \sum_{k=0}^{n-1} a_k & \text{Se } N > 0 \end{cases}$$



Recursividade - Exemplo

- Definição recursiva de $\sum_{k=0}^n a_k$

```
int somaVet(int *v, int n)
{
    if (n == 1)
        return v[0];
    else
        return v[n-1] + somaVet(v, n-1);
}
```



Recursividade - Exemplo

- Definição recursiva de $f(n)$

$$f(n) = \begin{cases} 3 & \text{Se } n = 0 \\ 2.f(n-1) + 3 & \text{Se } n > 0 \end{cases}$$

- Qual o valor de $f(4)$?

$$f(4) = 2.f(3) + 3 = ?$$

$$f(3) = 2.f(2) + 3 = ?$$

$$f(2) = 2.f(1) + 3 = ?$$

$$f(1) = 2.f(0) + 3 = ?$$

$$f(0) = 3$$

$$f(1) = 2.f(0) + 3 = 2.(3) + 3 = 9$$

$$f(2) = 2.f(1) + 3 = 2.(9) + 3 = 21$$

$$f(3) = 2.f(2) + 3 = 2.(21) + 3 = 45$$

$$f(4) = 2.f(3) + 3 = 2.(45) + 3 = 93$$



Sequência de Fibonacci

- Em 1202, Leonardo Fibonacci, na sua obra *Liber abaci*, propõe o seguinte problema:

Num pátio fechado coloca-se um casal de coelhos. Supondo que em cada mês, a partir do segundo mês de vida, cada casal da origem a um novo casal de coelhos, ao fim de um ano, quantos casais de coelhos estão no pátio?



Sequência de Fibonacci

- A sequência de Fibonacci é representadas pela seguinte função:

$$f(n) = \begin{cases} 0, & \text{se } n = 0, \\ 1, & \text{se } n = 1, \\ f(n-1) + f(n-2), & \text{se } n > 1. \end{cases}$$

- **Termos:** 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765, 10946, 17711, 28657, 46368, 75025, 121393, 196418, 317811, ...



Exemplo:Fibonacci

- Função que calcula o enésimo termo da sequência de Fibonacci, onde $n \in \mathbb{N}$.

```
long long fibrec(long long n)
{
    if (n<=1)
        return n;
    else
        return fibrec(n-1)+fibrec(n-2);
}

void main()
{
    long long fib, i=5;
    fib = fibrec(i);
    printf("fib(%d) = %d", i, fib);
}
```



Exemplo: Fibonacci

- Para valores pequenos, o programa é rápido. Já para $n > 40$ o computador requer um maior tempo de processamento.
- Verificando a função com mensagens de monitoramento

```
long long fibrec(long long n)
{
    long long F;
    printf("> Entrando em Fib(%d)\n", n);
    if (n<=1)
        F = n; /* caso base */
    else
        F = fibrec(n-1)+fibrec(n-2);
    printf("<<Saindo de Fib(%d), retorno=%d\n", n, F);
    return F;
}
```



Exemplo: Fibonacci

› Saída do monitoramento para $n = 4$:

› Entrando em Fib(4)

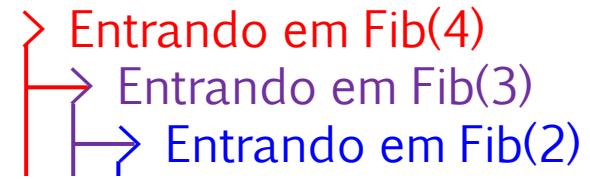
Exemplo: Fibonacci

› Saída do monitoramento para $n = 4$:

› Entrando em Fib(4)
└─> Entrando em Fib(3)

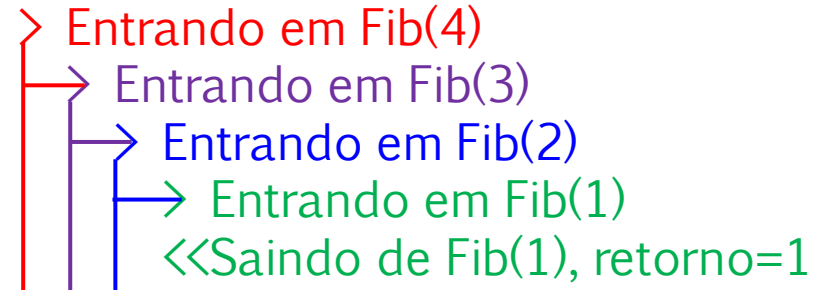
Exemplo: Fibonacci

› Saída do monitoramento para $n = 4$:



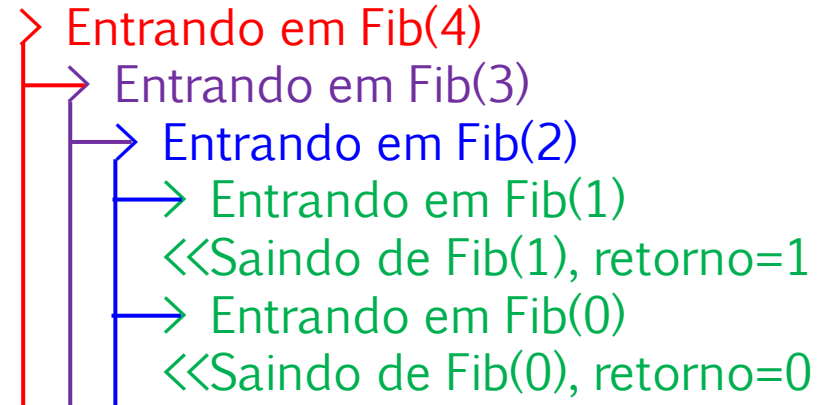
Exemplo: Fibonacci

› Saída do monitoramento para $n = 4$:



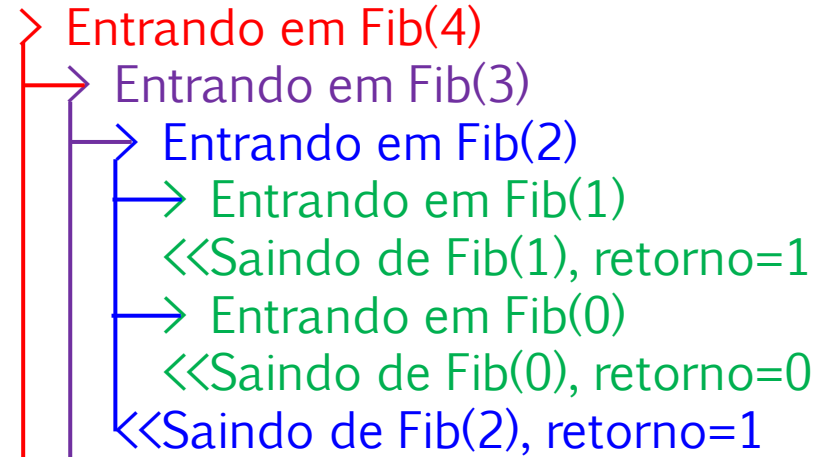
Exemplo: Fibonacci

› Saída do monitoramento para $n = 4$:



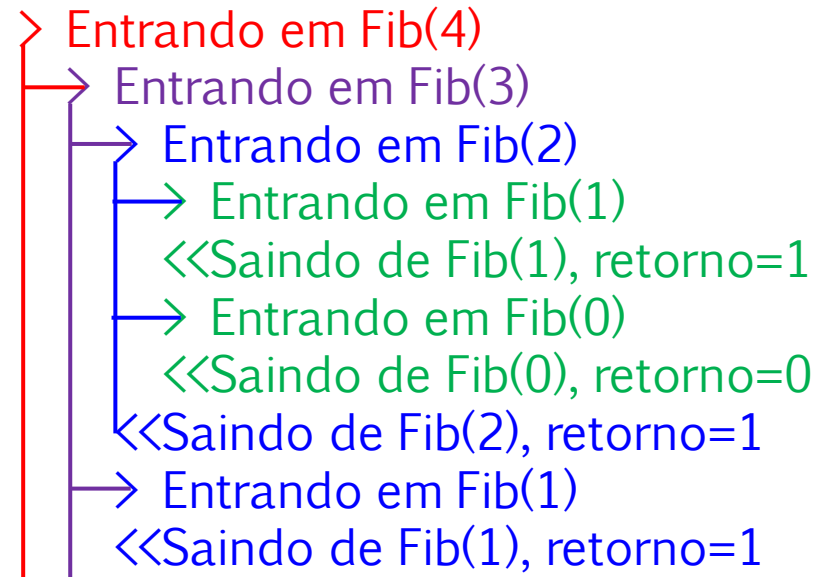
Exemplo: Fibonacci

› Saída do monitoramento para $n = 4$:



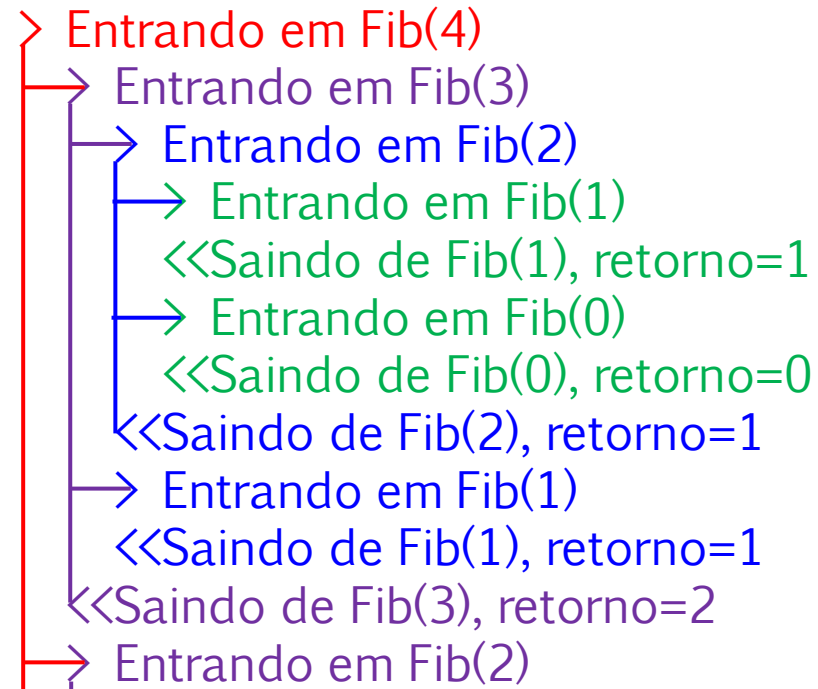
Exemplo: Fibonacci

› Saída do monitoramento para $n = 4$:



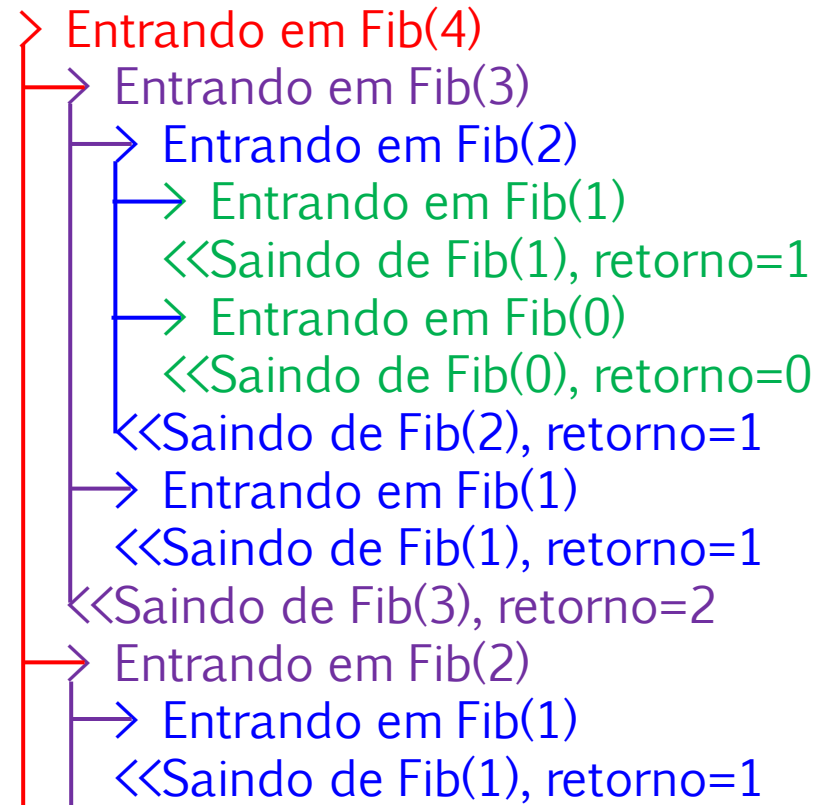
Exemplo: Fibonacci

› Saída do monitoramento para $n = 4$:



Exemplo: Fibonacci

› Saída do monitoramento para $n = 4$:



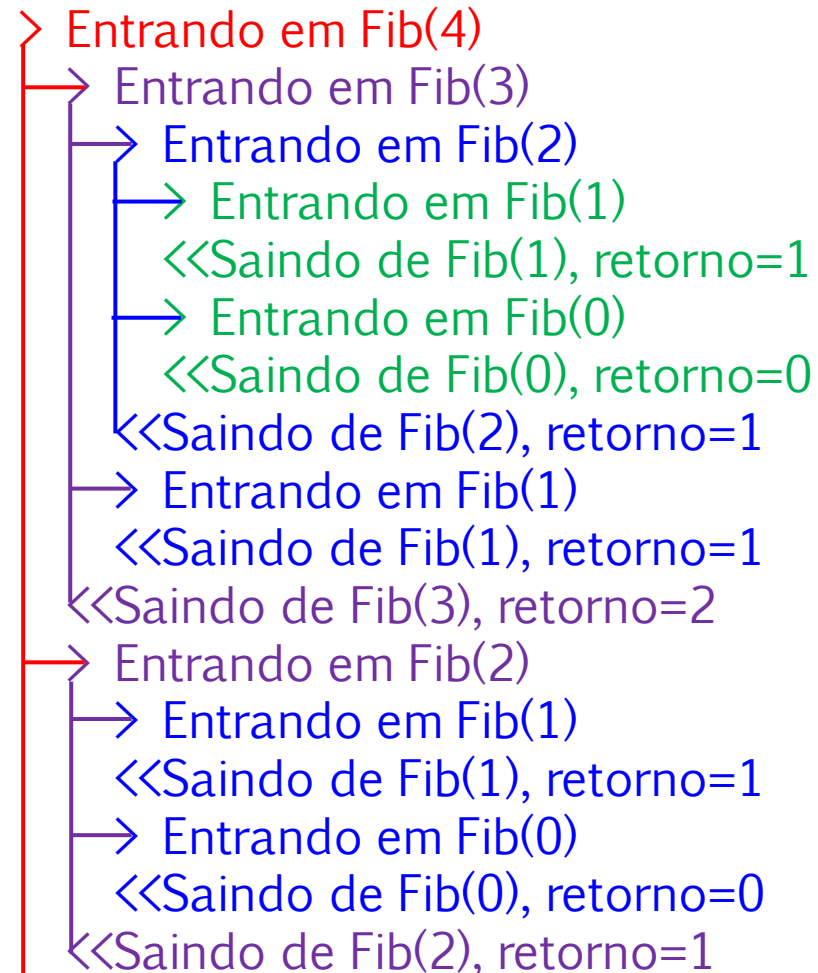
Exemplo: Fibonacci

› Saída do monitoramento para $n = 4$:



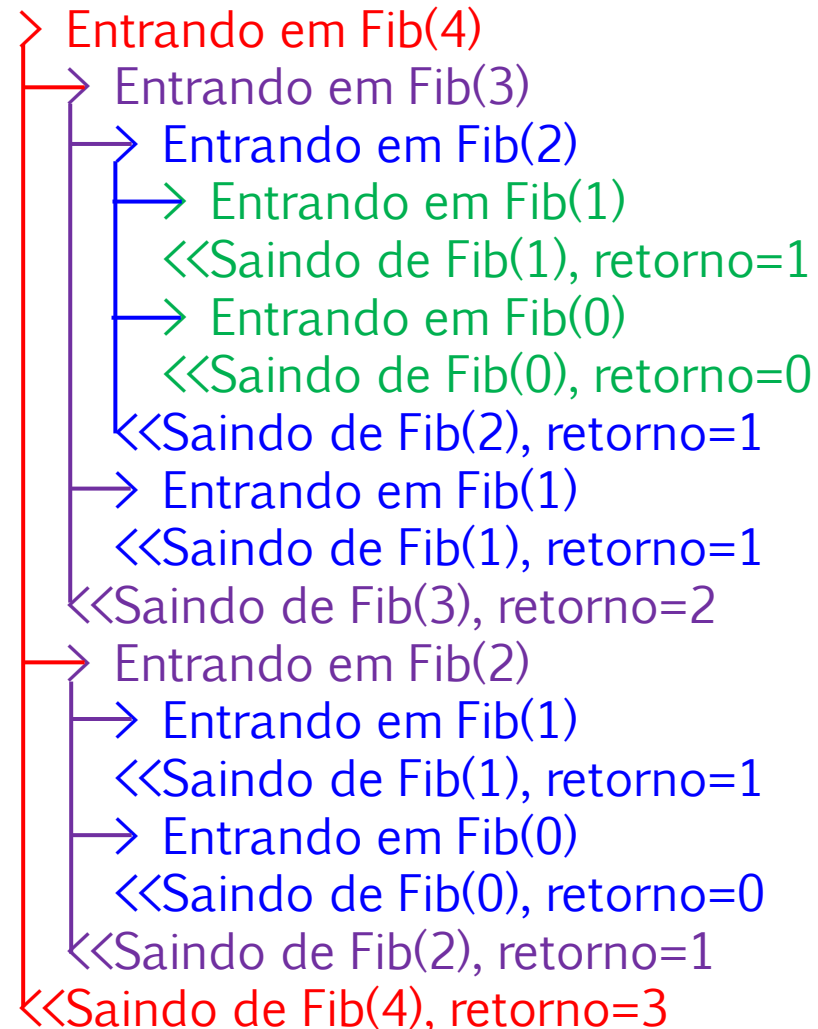
Exemplo: Fibonacci

› Saída do monitoramento para $n = 4$:



Exemplo: Fibonacci

› Saída do monitoramento para $n = 4$:



Sequência de Fibonacci

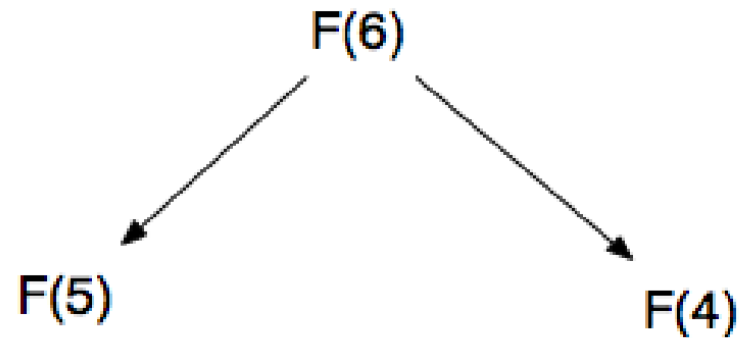
- $f(6) = ?$

F(6)



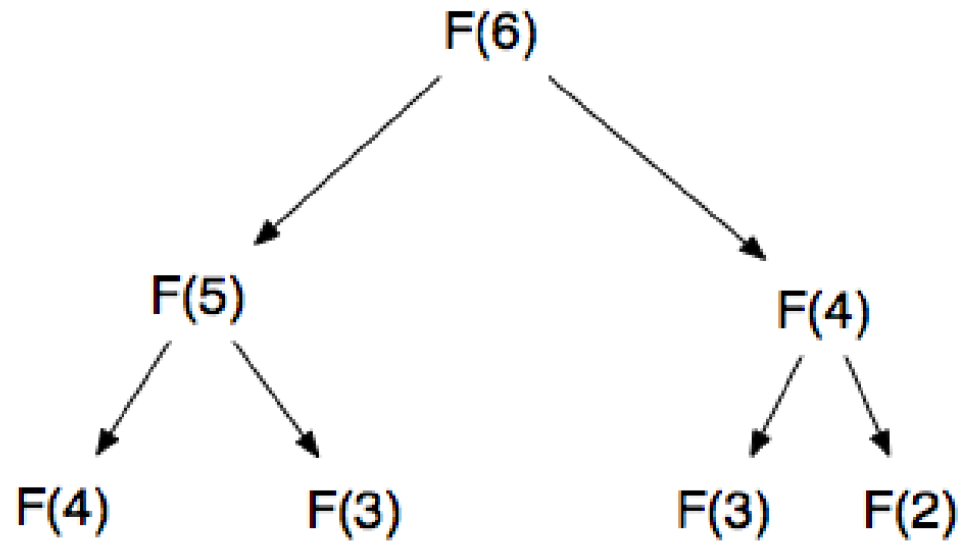
Sequência de Fibonacci

- $f(6) = ?$



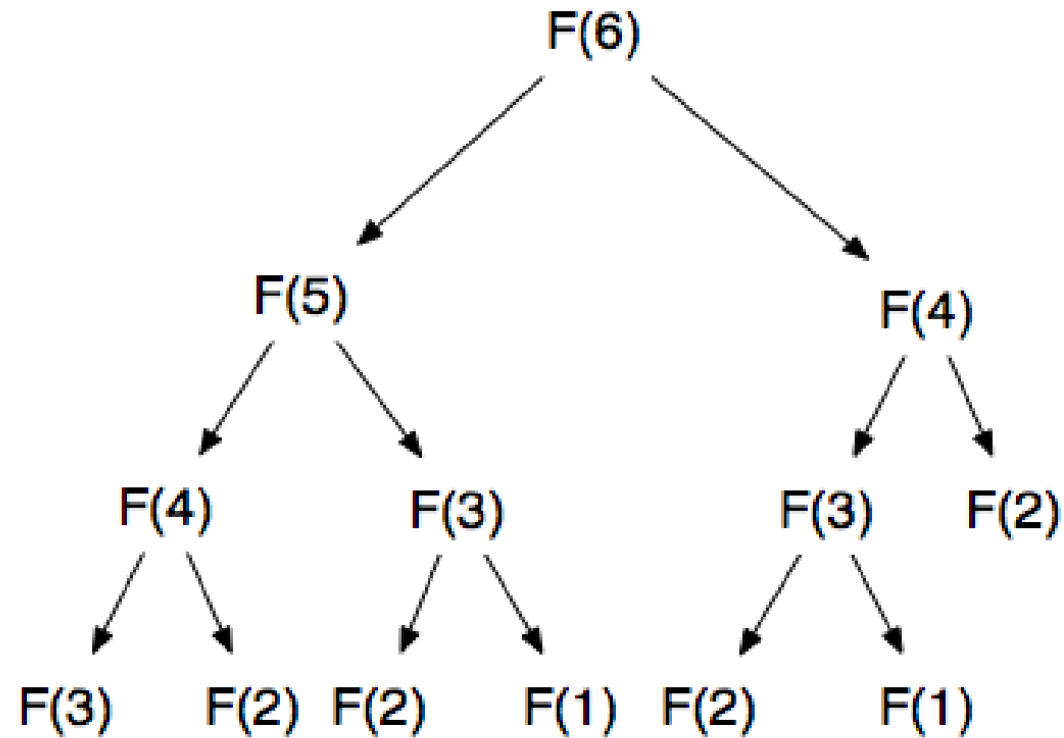
Sequência de Fibonacci

- $f(6) = ?$



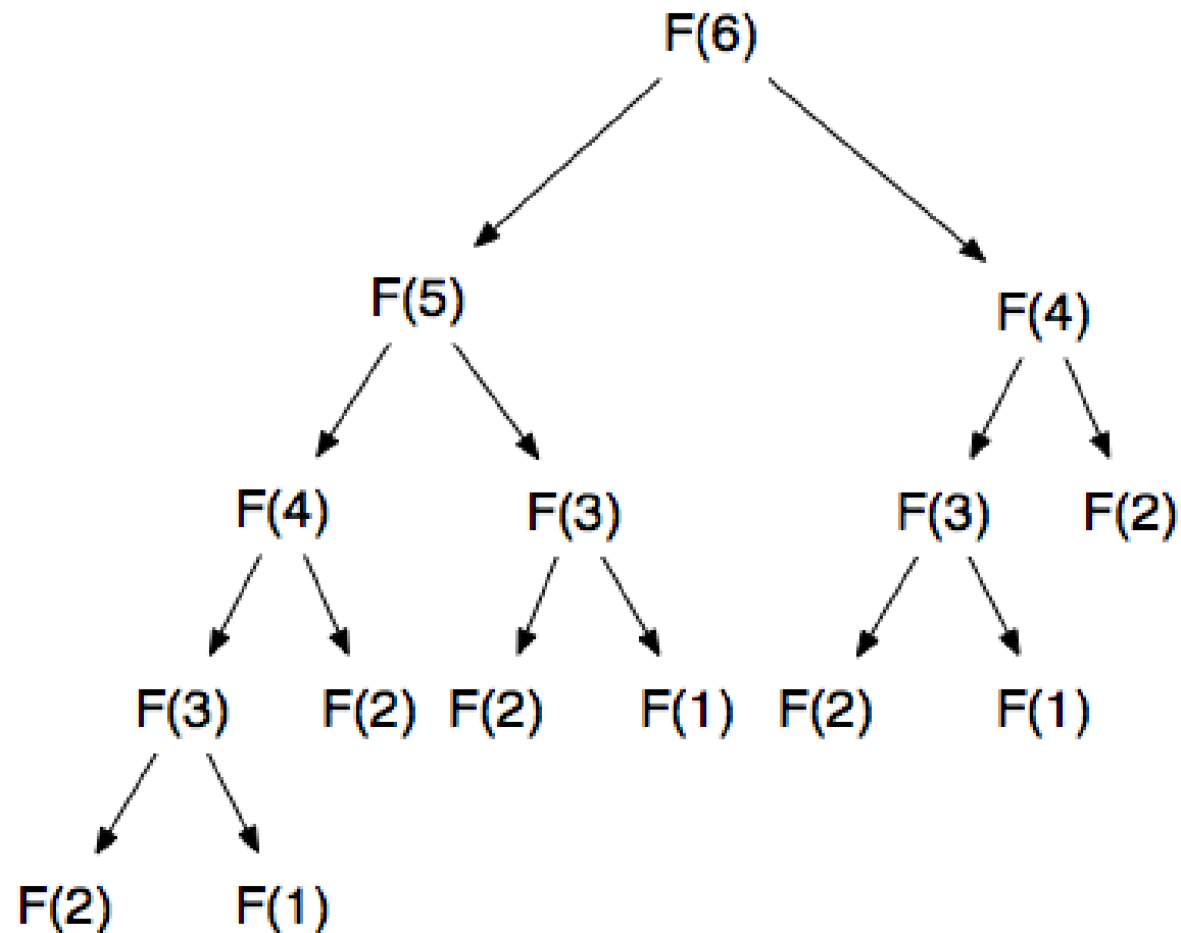
Sequência de Fibonacci

- $f(6) = ?$



Sequência de Fibonacci

- $f(6) = ?$



Cálculo do Fibonacci usando a proporção áurea

```
double gr = 1.618033988749895;

double FibGoldenRatio(int i)
{
    if (i == 1 )
        return 1;
    return round(gr * FibGoldenRatio(i - 1));
}
```



Função de Ackermann

- $$A(m, n) = \begin{cases} n + 1, & \text{se } m = 0 \\ A(m - 1, 1), & \text{se } m > 0 \wedge n = 0 \\ A(m - 1, A(m, n - 1)), & \text{se } m > 0 \wedge n > 0 \end{cases}$$

Qual o valor de $A(1, 2)$?



Função de Ackermann

```
int ack(int m, int n)
{
    if (m == 0){
        return n+1;
    }
    else if((m > 0) && (n == 0)){
        return ack(m-1, 1);
    }
    else if((m > 0) && (n > 0)){
        return ack(m-1, ack(m, n-1));
    }
}
```



Exercícios - Recursividade

1. Desenvolva algoritmos recursivos para os seguintes problemas:

- a) Escreva uma função recursiva para contar quantas vezes o caractere *c* ocorre na string *s*, onde o caractere e a string são fornecidos como parâmetro.
- b) Escreva uma função recursiva para computar a soma de todos os números de 1 a *n*, onde *n* é fornecido como parâmetro.
- c) Escreva uma função recursiva para determinar se um vetor de inteiro é um palíndromo, onde o vetor e seu tamanho são fornecidos como parâmetro.
- d) Escreva uma função recursiva para calcular a soma dos dígitos de um número inteiro e positivo, onde o número é fornecido como parâmetro. Por exemplo, a chamada da função para o valor 12345 deve retornar 15.
- e) Escreva uma função recursiva retornar um valor inteiro e positivo em ordem reversa, onde o valor é fornecido como parâmetro. Por exemplo, a chamada da função para o valor 123 deve retornar 321. Obs.: Não utilizar um vetor ou string para representar o valor.
- f) Escreva uma função recursiva para escrever em ordem os valores inteiros de *x* a *y*, onde *x* e *y* são fornecidos como parâmetro. Obs.: A lista a ser escrita pode estar em ordem crescente ou decrescente.
- g) Achar maior elemento de um vetor;
- h) Achar um elemento em um vetor ordenado de comprimento potência de 2.



Exercícios - Recursividade

2. ROSEN, K. H. **Matemática discreta e suas aplicações**. 6ª ed., São Paulo: McGraw-Hill, 2009.

- Seção 4.3
- 1, 2, 3, 4, 5, 6, 9

